Chapter 1

Advanced Algorithms

Area of Study 2: Advanced algorithm design

Learning Intentions

- **Key knowledge**
  - tree search by backtracking and its applications
  - the application of heuristics and randomised search to overcoming the soft limits of computation, including the limitations of these methods
  - hill climbing on heuristic functions, the A* algorithm and the simulated annealing algorithm
  - the graph colouring, 0-1 knapsack and travelling salesman problems and heuristic methods for solving them
- **Key skills**
  - apply the divide and conquer, dynamic programming and backtracking design patterns to design algorithms and recognise their usage within given algorithms
  - develop different algorithms for solving the same problem, using different algorithm design patterns, and compare their suitability for a particular application
  - apply heuristics methods to design algorithms to solve computationally hard problems
  - explain the application of heuristics and randomised search approaches to intractable problems, including the graph colouring, 0-1 knapsack and travelling salesman problems

Classic Problems

Now that we have considered the limits of computation, we can look at some classic problems that illustrate these limits and how advanced algorithms can help find solutions. Each of these problems has a **decision version** and an **optimisation version**. The **decision version** asks whether a solution exists that meets certain criteria, while the **optimisation version** seeks the best solution according to some metric.

Optimisation problem:

- Goal: Find the best solution according to some objective.
- Output: The actual solution and/or its optimal value.
- Nature: Search for the best among many possible solutions.

Decision problem:

- Goal: Answer yes/no — does a solution exist that meets a certain condition?
- Output: Boolean (yes or no).
- Nature: Confirm if a feasible solution exists that satisfies the constraint.

Why the distinction?

Decision problems are often easier to solve than optimisation problems because they require only a yes/no answer rather than finding the best solution.

You can transform an optimisation problem into a decision problem by introducing a threshold (or target value) for the objective function and asking whether there exists a feasible solution that meets or exceeds this threshold.

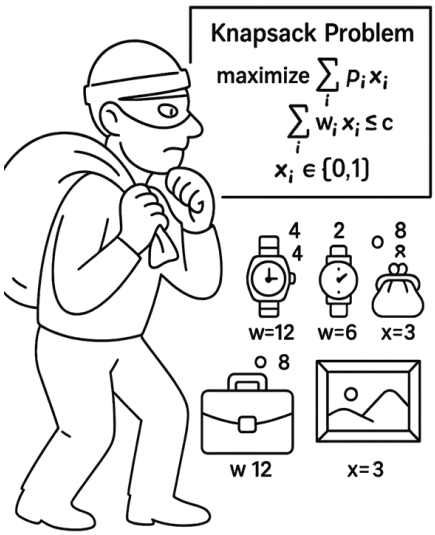
If you have an algorithm for the decision version, you can use it to solve the optimisation version by searching over possible thresholds (linear or binary search) to find the maximum (or minimum) value for which the answer is "yes". This approach can reduce the search space significantly and make the problem more manageable—particularly when the decision version can be solved in polynomial time.

Graph Colouring

- **Optimisation problem**: What is the minimum number of colours needed to colour a graph such that no two adjacent vertices share the same colour?
- **Decision problem**: Can a graph G be coloured with at most k colours so that no two adjacent vertices share a colour?
- **Complexity**:
  - Decision problem is **NP-complete** for $k \geq 3$.
  - Optimisation version is **NP-hard**.
  - Special cases (e.g., $k = 2$) solvable in polynomial time.
- **Applications**:
  - Scheduling (e.g., timetabling exams without conflicts).
  - Register allocation in compilers.
  - Frequency assignment in wireless networks.
- **Origins**:
  - Originates from the **Four Colour Problem** (1852, Francis Guthrie), asking whether four colours suffice to colour any map so that no adjacent regions share a colour.
  - Later reformulated in graph theory terms as a vertex colouring problem.
  - Four Colour Theorem proved in 1976 by Appel and Haken using computer assistance.

0–1 Knapsack

- **Optimisation problem**: Select a subset of items, each with a value and weight, to maximise total value without exceeding the weight capacity. Each item can be taken (1) or left (0).
- **Decision problem**: Given a set of items with values and weights, a maximum capacity W, and a target value V, is there a subset whose total weight $\leq W$ and total value $\geq V$?
- **Complexity**:
  - Decision version is **NP-complete**.
  - Optimisation version is **NP-hard**.
  - Solvable in $O(nW)$ via dynamic programming (pseudo-polynomial time).
- **Applications**:
  - Budget allocation.
  - Cargo loading.
  - Project selection with resource limits.
  - Investment portfolio optimisation.
- **Origins**:
  - Studied in the 19th and early 20th centuries in the context of resource allocation problems.
  - Became a classic combinatorial optimisation problem in operations research by mid-20th century.
  - Named "knapsack" because of the analogy to packing items in a backpack.



Travelling Salesman Problem (TSP)

- **Optimisation problem**: Find the shortest possible tour that visits each city exactly once and returns to the starting city.
- **Decision problem**: Given a set of cities, distances between them, and a maximum length D, is there a tour visiting each city exactly once with total length $\leq D$?
- **Complexity**:
  - Decision version is **NP-complete**.
  - Optimisation version is **NP-hard**.
  - Exact algorithms are exponential (e.g., Held–Karp DP in $O(n^2 2^n)$).
- **Applications**:
  - Route planning and logistics.
  - Manufacturing (e.g., circuit board drilling).
  - Genome sequencing.
  - Data clustering.
- **Origins**:
  - Dates back to 18th-century mathematical puzzles (e.g., Hamiltonian cycles in 1850s).
  - Named and popularised in the mid-20th century by operations research and the RAND Corporation.
  - Studied extensively as a benchmark for combinatorial optimisation and computational complexity.

0–1 Knapsack Algorithms

KnapsackBruteForce

```
1: procedure KnapsackBruteForce(W, V, C, i)
                                                          ▷ W = weights
2:                                                          ▷ V = values
3:                                                       ▷ C = capacity,
4:                                                          ▷ i = index
5:
6:     if i = length(W) then
7:         return 0
8:     end if
9:     best ← KnapsackBruteForce(W, V, C, i + 1)                ▷ Skip item i
10:    if W[i] ≤ C then
11:        includeValue ← V[i]+ KnapsackBruteForce(W, V, C − W[i], i + 1)
12:        if includeValue > best then
13:            best ← includeValue
14:        end if
15:    end if
16:    return best
17: end procedure
```

KnapsackBruteForceBitmask

```
1: procedure KnapsackBruteForceBitmask(W, V, C)
2:     n ← length(W)
3:     bestValue ← 0
4:     for mask ← 0 to 2^n − 1 do                          ▷ Loop over all subsets
5:         totalW ← 0
6:         totalV ← 0
7:         for i ← 0 to n − 1 do
8:             if mask[i] = 1 then
9:                 totalW ← totalW + W[i]
10:                totalV ← totalV + V[i]
11:            end if
12:        end for
13:        if totalW ≤ C then
14:            bestValue ← max(bestValue, totalV)
15:        end if
16:    end for
17:    return bestValue
18: end procedure
```

1.1 Exercise

1. Trace the execution of the algorithms
2. Modify the algorithms to answer the decision version of the problem

Dynamic Programming

Invented by Richard Bellman in the 1950s. Dynamic programming is an algorithm design technique that combines brute force and greedy ideas. It is used to solve problems by breaking them down into simpler subproblems and storing the results of these subproblems to avoid redundant computations.

It makes use of:

- Recursion + dictionary (top-down memoization)
- Iteration (bottom-up tabulation)

An algorithm design technique for problems that have:

- **overlapping subproblems** (the same subproblems recur), and
- **optimal substructure** (an optimal solution is built from optimal subsolutions).

Key idea: solve each subproblem once, store the result in a table or dictionary, and reuse it to avoid recomputation.

Two main styles:

Top-down (memoization):

- write the natural recursive solution
- add a memo table to cache results as they are computed

Bottom-up (tabulation):

- identify the subproblem order from smallest to largest
- fill a table iteratively until the target answer is reached

Example: Fibonacci

Definition: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$

Naive recursion (not DP)

```
function Fib(n):
  if n <= 1: return n
  return Fib(n-1) + Fib(n-2)
```

1. What is the time complexity of this function?

---

2. Draw the recursion tree for Fib(5)

This function is inefficient because it repeats the same calculation many times.

3. How many redundant function calls are made when fib(5) is called?

Recursion with Memoisation

```
memo = {}
function Fib(n):
  if n in memo: return memo[n]
  if n <= 1: return n
  memo[n] = Fib(n-1) + Fib(n-2)
  return memo[n]
```

Recursion with Memoisation

```
memo = {0: 0, 1: 1}

function FibM(n):
  if n in memo: return memo[n]
  memo[n] = FibM(n-1) + FibM(n-2)
  return memo[n]
```

4. What is the time complexity of FibM?

Iteration Bottom UP

Full table (classic bottom-up DP)

```
function FibTable(n):
  dp[0] = 0
  dp[1] = 1
  for i from 2 to n:
  dp[i] = dp[i-1] + dp[i-2]
  return dp[n]
```

- Stores **all values** $F(0) \dots F(n)$.
- Time O(n), space O(n).

Optimised bottom-up (rolling two values)

The "table" doesn't have to be an array — it can be a dictionary, or even just a couple of variables, as long as you're storing and reusing subproblem results.

```
function FibOptimised(n):
  if n <= 1: return n
  a = 0; b = 1       # F(0), F(1)
  for i from 2 to n:
  temp = a
  a = b
  b = temp + b
  return b
```

- Stores **only two values**: $F(i-2)$ and $F(i-1)$.
- Time O(n), space O(1).
- Same logic, just no big table.

Bottom-up and DP

- **Bottom-up DP:**
  - For problems with overlapping subproblems + optimal substructure
  - Builds solutions from small subproblems up to the final answer
  - Stores results (full table or compressed version) → avoids recomputation
  - Inherently **dynamic programming**
- **But not all bottom-up algorithms are DP**
  - Example: **Insertion sort**
    - Builds sorted array one element at a time
    - No reuse of subproblem results → not DP

Exercise

Consider the change-making problem with available coin denominations $\{1, 3, 5\}$. Let $dp[x]$ denote the minimum number of coins needed to make amount x.

Recurrence (problem structure)

$$dp[0] = 0, \qquad dp[x] = 1 + \min\big(dp[x-1],\, dp[x-3],\, dp[x-5]\big) \text{ for } x \geq 1,$$

ignoring any term with a negative index.

1. Write a recursive function that computes the minimum number of coins to make amount x using the recurrence above.
2. Draw the recursion tree for your recursive function when $x = 7$.
3. Modify your recursive algorithm to use a dictionary `memo` so that each subproblem x is solved at most once.
4. For $x = 7$, how many recursive calls are saved?
5. After computing $x = 7$ with your memoized algorithm, list the contents of `memo` (i.e., the x values that were cached and their $dp[x]$ values).
6. **Iterative algorithm.** Write an iterative bottom-up algorithm that fills $dp[0 \ldots n]$ and returns $dp[n]$.
7. Question 10 (VCAA2023)

   A dynamic programming algorithm is used to solve a change-making problem with the coin denominations of 1, 3 and 4. This algorithm iteratively solves the sub-problem of finding the fewest coins required to give k change. The algorithm is used to solve the problem for a total amount of 6.

   What are the final values stored in the array used by the algorithm? A. [0, 1, 2, 1, 1, 2, 2]

   B. [0, 1, 1, 3, 4, 4, 3]

   C. [0, 1, 1, 3, 4, 1, 3]

   D. [3, 3]
8. Question 20 (VCAA2019)

   Which one of the following descriptions of dynamic programming and divide and conquer is correct?

   A. Dynamic programming aims to solve the sub-problems once, whether they are overlapping or not, whereas divide and conquer does not care about how many times it needs to solve a sub-problem.

   B. Divide and conquer aims to solve the sub-problems once, whether they are overlapping or not, whereas dynamic programming does not care about how many times it needs to solve a sub-problem.

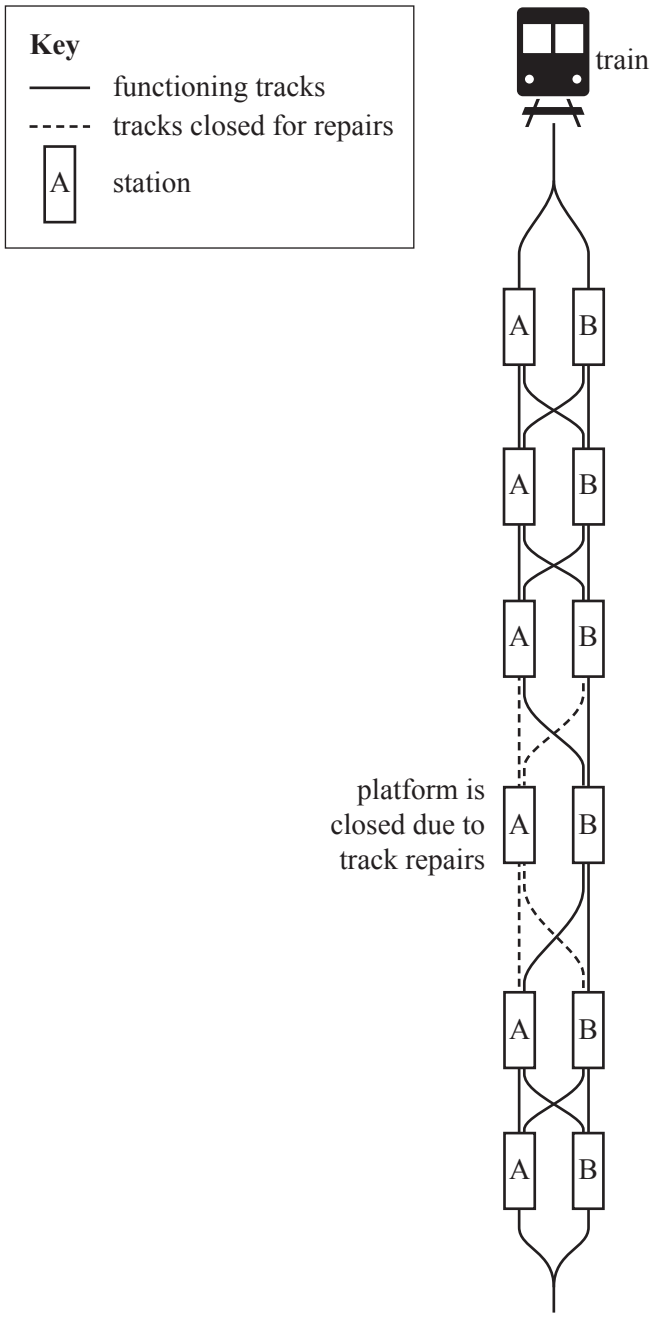
   C. Dynamic programming aims to find an optimal solution for a problem by splitting it into non-overlapping sub-problems, finding the optimal solutions for the sub-problems, and combining the optimal solutions for the sub-problems to form the optimal solution for the original problem.

   D. Divide and conquer aims to find an optimal solution for a problem by splitting it into overlapping sub-problems, finding the optimal solutions for the sub-problems with the intention of solving the overlapping sub-problems only once, and combining the optimal solutions for the sub-problems to form the optimal solution for the original problem.

Question 6 (7 marks)

A metropolitan train company has asked Maia to assist with scheduling trains travelling along a train network. Each station along the network has two platforms and interconnecting tracks. The expected wait time at each platform and the time taken to travel along that track depend on the number of staff allocated to assist with boarding and signalling. At times, platforms or tracks may be closed for repairs.

The proposed network is modelled below.



a. One approach to help with scheduling is to use a brute-force algorithm to reduce congestion for each train travelling through the network.

Explain whether or not this is feasible. Include the time complexity in your explanation. 4 marks

b. Maia suggests that a dynamic programming approach should be used for scheduling as the train network is likely to expand.

What properties of this problem make it suitable for a dynamic programming approach? 3 marks

Question 9 (10 marks)

A Melbourne property developer would like to buy a stretch of land to build some townhouses. She has identified a street on which she would like to build and has spoken to each of the property owners on the street to determine how much profit she would make by building on their land.

She has stored this information in an array, with each of the elements in the array representing the profit (or loss) for the purchase, in thousands of dollars.

For example, the array P = [80, −120, 50] would represent three houses, where buying the first would give the developer $80 000 in profit, buying the second would result in a $120 000 loss and buying the third would give a $50 000 profit.

The developer would like to solve the problem of determining the greatest profit she could make by buying a single, continuous stretch of land.

Three examples of continuous stretches of land with the greatest profit are provided below.

P = [80, −120, 50] : greatest profit is 80 thousand. ([**80,** −120, 50])

P = [20, −40, 90, 20, −50, 70] : greatest profit is 130 thousand. ([20, −40, **90, 20, −50, 70**])

P = [25, −10, −20, 50, 40, −30] : greatest profit is 90 thousand. ([25, −10, −20, **50, 40,** −30])

a. Describe a brute-force approach for solving this problem. Do **not** provide pseudocode in your answer. 3 marks

b. Discuss whether a divide and conquer approach could be used to solve this problem. Justify your answer. 3 marks

c. Write pseudocode for a dynamic programming algorithm that would solve this problem. 4 marks

Backtracking

Backtracking: Systematic Search with Pruning

DFS explores all possibilities down to the leaves. **Backtracking** is DFS *plus pruning*: it abandons a partial solution as soon as it cannot possibly lead to a valid complete solution.

- DFS on a graph: visits every node/edge eventually.
- Backtracking on a search tree: **prunes** branches early.
  - Sudoku: stop when a number violates a row/column/block.
  - Word search: stop when partial path doesn't match the target prefix.

General Backtracking Pattern:

Backtrack

```
1: procedure Backtrack(state)
2:     if IsSolution(state) then
3:         Process(state)
4:         return
5:     end if
6:     for each choice in Options(state) do
7:         if IsValid(choice, state) then
8:             Make(choice, state)
9:             Backtrack(state)
10:            Unmake(choice, state)
11:        end if
12:    end for
13: end procedure
```

Key modification points:

- IsSolution: when a complete/valid solution is reached.
- IsValid: prune invalid partial solutions early.
- Process: collect/count solutions, update best-so-far.
- Make/Unmake: modify/restore the state.

Example: Word Search

Suppose we want to find the word "LIFE" in a grid or graph. Backtracking explores all possible paths, pruning those that do not match the target prefix.

State

Is Solution

for choice in OPTIONS

Is Valid

Make

Unmake

Word search algorithm

Warm up

1. Write **Prim's algorithm** as concisely as possible.
2. Write **Dijkstra's algorithm** as concisely as possible.
3. Trace Dijkstra's on the given graph assuming we want the shortest path from P to R. Mark nodes as they are explored.

A* Algorithm

A **heuristic** is an additional piece of information that can be used to solve a problem.

A* is a search algorithm to find the shortest path from a start node to a goal node.

It combines **Dijkstra's** and a **heuristic**.

What information could we use to make our search more efficient?

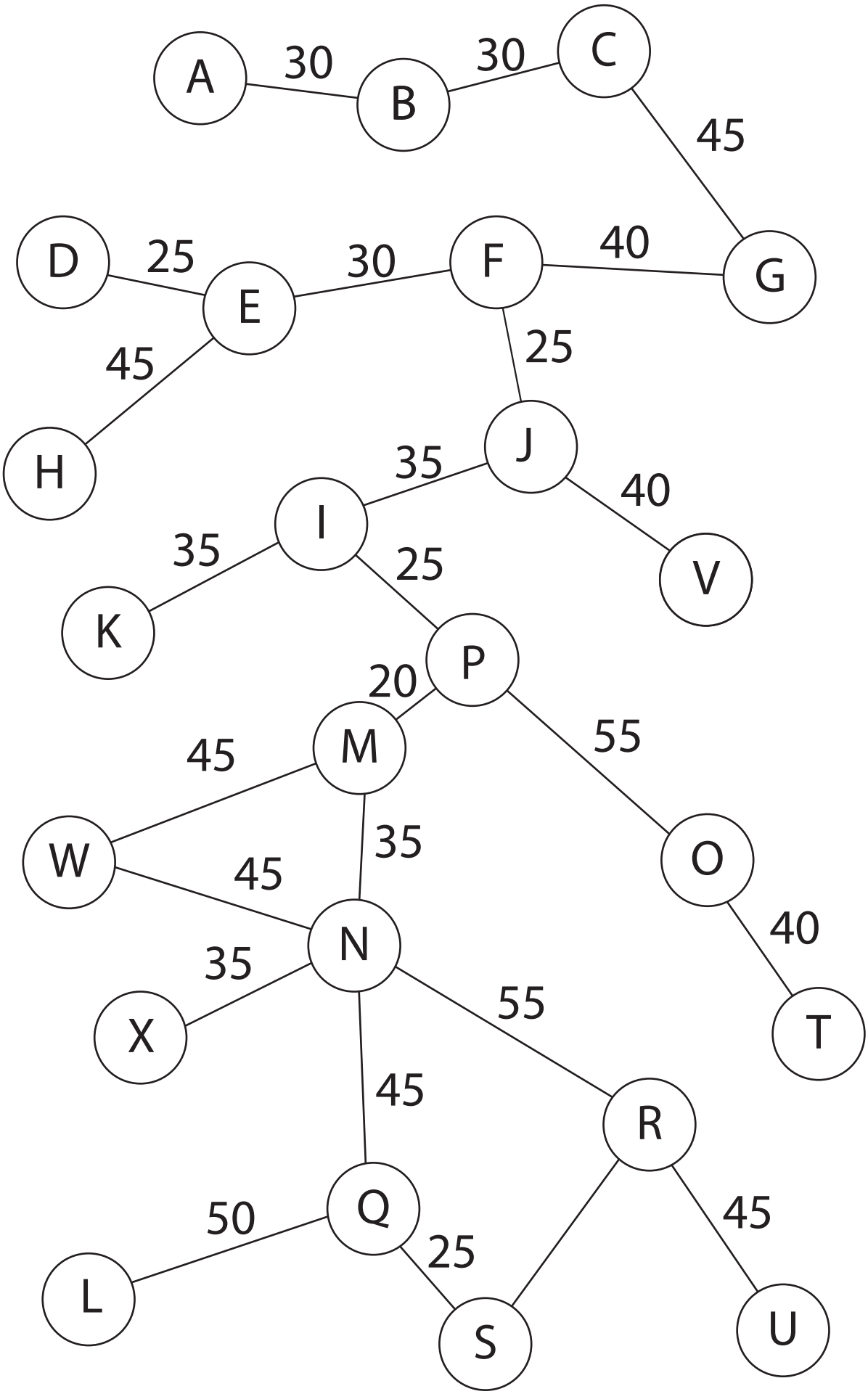
- Dijkstra's explores by actual path cost so far ($g(n)$).
- A* uses the actual cost plus heuristic cost ($f(n)$).

A*: $f(n) = g(n) + h(n)$

where:

- $g(n)$ is the actual cost
- $h(n)$ is the heuristic cost

Run A* on the given graph. Mark the nodes explored by A*.


A
30
B
30
C
45
D
25
E
30
F
40
G
45
25
H
J
35
40
I
35
25
V
K
P
20
M
45
55
W
35
45
O
40
N
35
55
X
T
45
R
Q
50
45
25
L
S
U

A* Algorithm (Pseudocode)

Modify Dijkstra's to use A*.

Assume you have a list h that contains the heuristic values for each node, e.g. $h[A] = 4$, $h[B] = 2$, $h[C] = 0$.

Dijkstra

```
1:  procedure DIJKSTRA(G, s)
2:      for each v in G do
3:          dist[v] ← ∞
4:          parent[v] ← NIL
5:      end for
6:      dist[s] ← 0
7:      Q ← {(s, 0)}
8:      while Q not empty do
9:          u ← extract_min(Q)
10:         for each (v, w) in Adj[u] do
11:             if dist[u] + w < dist[v] then
12:                 dist[v] ← dist[u] + w
13:                 parent[v] ← u
14:                 update(Q, v, dist[v])
15:             end if
16:         end for
17:     end while
18:     return dist, parent
19: end procedure
```

Heuristics

Rule of thumb: guides the search process. Trade **guaranteed optimality** for **speed**.

Why Heuristics?

- Some problems are **NP-hard** (e.g., TSP, Knapsack, Graph Colouring)
- Exact solutions are often **infeasible** for large input sizes
- Need strategies that are **good enough** and run in **reasonable time**

Properties of Heuristics

- **Admissible**: never overestimates cost (ensures optimality in A*)
- **Consistent**: satisfies triangle inequality (ensures finality of settled nodes)
- **Domain-specific**: often tailored to the problem
- **Limitations**: may miss optimal solution, may still be slow

Heuristics Examples

1. **Exam Timetabling**
   - **Neighbourhood**: A timetable (move = swap two exams or move one exam to a different slot)
   - **Problem**: Scheduling university exams so that no student has two exams at the same time
   - **Heuristic**: _____
2. **Delivery Routes**
   - **Neighbourhood**: A delivery route (move = swap the order of two addresses, or insert one at a different point)
   - **Problem**: A courier must deliver parcels to 50 addresses and return to the depot
   - **Heuristic**: _____
3. **Job Scheduling**
   - **Neighbourhood**: A job sequence (move = swap the order of two jobs, or shift one earlier/later in the sequence)
   - **Problem**: A machine shop has jobs of different lengths; the goal is to minimise the average completion time
   - **Heuristic**: _____

Heuristics for NP-hard Problems

Suggest suitable heuristics for the following NP-hard problems:

- **Travelling Salesman Problem (TSP)**: _____
- **Knapsack Problem**: _____
- **Graph Coloring**: _____

Hill Climbing

An algorithmic technique that relies solely on local heuristics.

```
At each step, it looks at one neighbour (sometimes chosen at random).
If that neighbour has a better heuristic value (e.g. closer to the goal), move there.
If not, stay put.
Repeat until the goal is reached or you get stuck.
```

Hill Climbing

```
current = start
while current != goal:
  neighbor = select_random_neighbor(current)
  if h(neighbor) < h(current):
    current = neighbor
return current
```

Hill Climbing

- Imagine finding the highest peak in a hilly terrain by always moving uphill.
- Hill climbing is a **metaphor**, you can use other heuristics
- Example:
  - Move through a grid to some goal
  - Find the maximum x value of a function $f(x) = -x^2 + 5x$

Hill Climbing in Search

Hill Climbing can be used to **search through a solution space**.

Neighbors are similar solutions, typically differing by a small change.

Examples of Hill Climbing

Exam Timetabling

- **Problem**: Reduce clashes between students.
- **Heuristic**: Number of clashes in the timetable.
- **Rule**: Swap two exams if it reduces clashes.

Travelling Salesman Problem (TSP)

- **Problem**: Find a short tour through all cities.
- **Heuristic**: Total length of the tour.
- **Rule**: Swap two cities if it produces a shorter tour.

Knapsack Approximation

- **Problem**: Maximise value of items under weight capacity.
- **Heuristic**: Total value/weight ratio of the current selection.
- **Rule**: Add an item if it increases the ratio; otherwise, don't.

Sudoku Solver

- **Problem**: Fill a 9×9 Sudoku grid.
- **Heuristic**: Number of rule violations (duplicate numbers in rows, columns, boxes).
- **Rule**: Change a cell only if it reduces violations.

Key Idea of Hill Climbing

- Start with a **current solution**.
- Measure it with a **heuristic**.
- Make a **small local change** that improves the heuristic.
- Stop when no further improvement is possible.

Problems

- Can get stuck in local optima.
- Can get trapped in plateaus (flat areas in the landscape).
- Outcome depends on initial conditions.
- Requires a good heuristic to guide the search.

Fixes

- Use **random restarts**: If stuck, restart from a different initial solution.
  - repeat k times and pick the best result.
- **Combine with Other Heuristics (Metaheuristics)**
  - Use multiple heuristics or higher-level strategies to guide the search.
  - Example: simulated annealing

Variations of Hill Climbing

- **Steepest-Ascent**: check *all* neighbours, move to the best one
- **Stochastic**: pick a random neighbour, move if it improves
- **First-Choice**: scan neighbours in given order, stop at the first improvement
- All are greedy: only move if the heuristic improves

Simulated Annealing

> "Sometimes you have to take two steps back to take ten forward."
>
> *Nipsey Hussle*

Simulated annealing is a heuristic optimisation algorithm that introduces randomness into the search process. Unlike hill climbing, which only accepts improvements, simulated annealing sometimes accepts worse solutions with a probability that decreases over time. This allows the algorithm to escape local maxima or minima and increases the chance of finding a global optimum.

The method is inspired by metallurgical annealing: when heated, atoms can move freely into many configurations; as the material cools, they settle into a stable, low-energy state. Similarly, simulated annealing begins by exploring widely, even accepting poor moves, but gradually becomes more selective as the "temperature" decreases.

The heuristic value of a solution is defined as its energy, with lower energy representing a better outcome. The algorithm aims to minimise this energy, settling on a near-optimal solution as the system "freezes."

Algorithm Steps

The process of simulated annealing can be summarised as follows:

1. Begin with an initial solution.
2. At each step, select a neighbour solution.
   - If the neighbour has lower energy than the current solution, it is always accepted.
   - If the neighbour has higher energy, it may still be accepted with probability

     $$P = e^{-\Delta E/T}, \quad \Delta E = E(\text{new}) - E(\text{current})$$

     where $E(x)$ is the energy (or cost) of solution x, and T is the current "temperature."
3. Decrease the temperature over time
4. Repeat until frozen, leaving the algorithm with a stable, low-energy solution.

Simulated Annealing

```
current ← initial solution
T ← initial temperature
while system not frozen do
    pick a random neighbor
    ΔE ← E(neighbor) − E(current)
    if ΔE < 0 then
        current ← neighbor                    ▷ always accept better (lower energy)
    else
        P ← e^(−ΔE/T)
        r ← random number in [0, 1]
        if r < P then
            current ← neighbor                ▷ sometimes accept worse
        end if
    end if
    decrease T
end while
return current                                ▷ final low-energy solution
```

Why it works

- **Hill Climbing**: only accepts better moves → gets stuck in local optima.
- **Simulated Annealing**: sometimes accepts worse moves, especially at the start (high T).
- As T lowers, it becomes more like hill climbing (only better moves).
- This balance lets it **escape local optima** and explore more of the solution space.

Example: Travelling Salesman Problem (TSP)

Heuristic: $E(\text{tour})$ = total distance of the tour.

Lower distance = better tour.

Neighbour solution = swap two cities in the order.

1. Start with a random tour of cities.
2. Swap two cities.
   - If distance decreases → accept.
   - If distance increases → accept with probability that decreases as T decreases.
3. Early in the run: lots of exploration.
4. Later: fine-tuning near the best tours.

Temperature

The temperature affects the probability of accepting a worse move. After each iteration the temperature T is reduced. The most common method is **geometric cooling**:

$$T \leftarrow \alpha T$$

with $0 < \alpha < 1$ (typically $\alpha \approx 0.9$–0.99). A slower cooling rate (closer to 1) allows more exploration, while a faster rate makes the search more greedy.

$\Delta E/T$	Acceptance Probability $P = e^{-\Delta E/T}$
0.1	0.90
0.5	0.61
1	0.37
2	0.14
5	0.007

Table 1.1: Acceptance probability for different values of $\Delta E/T$.

The algorithm must eventually stop once the system is considered "frozen". Typical terminating conditions include:

- **Fixed number of iterations** – the algorithm runs for a predetermined number of steps.
- **Convergence** – stop when the solution does not improve (or does not change) for a set number of iterations.
- **Minimum temperature** – stop when the temperature T falls below a small threshold $T_{\min}$.

Properties

- **Metaheuristic**: a higher-level strategy for guiding heuristics.
- **Does not guarantee optimality**, but often finds *near-optimal* solutions in large search spaces.
- Useful for **NP-hard problems**: TSP, timetabling, layout optimisation, etc.

1.2 Exercise

1. Question 17 2019

   Which of the following properties, where intensification narrows the search to a local region and diversification considers other regions of the search space, is more likely to cause convergence towards

   A. intensification by itself

   B. diversification by itself

   C. both intensification and diversification

   D. neither intensification nor diversification

2. Question 17 2020

   Which one of the following is not a limitation of heuristic algorithms?

   A. Some of the solution space may not be considered.

   B. The solution returned may not be optimal.

   C. They may be too slow to be useful.

   D. The solution may not converge on a good result.

3. Question 18 2022

   Which one of the following describes the method of a greedy heuristic algorithm?

   A. Select a local optimum in each step in the hope of progressing towards a global optimum solution.

   B. The problem is divided into smaller sub-problems and their solutions are combined to form the solution to the original problem.

   C. Generate candidates from the solution space in the hope of finding the global optimum solution.

   D. Randomly select one option from several candidates in each step, hoping to eventually obtain a global optimum solution.

   Question 1 (3 marks) 2016 Explain how randomised heuristics can help overcome the soft limits of computation. Use an example as part of your explanation.

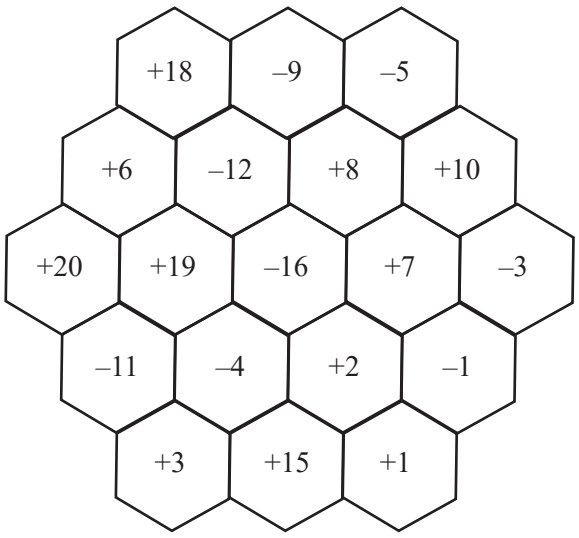
Question 12 (11 marks)

HexaReverso is a single-player game played with hexagonal tiles. Each tile has an integer value, with one side having a positive value and the reverse side having the negative of that value. The tiles are arranged in a hexagonal shape. Large hexagonal grids can be hundreds of tiles wide.

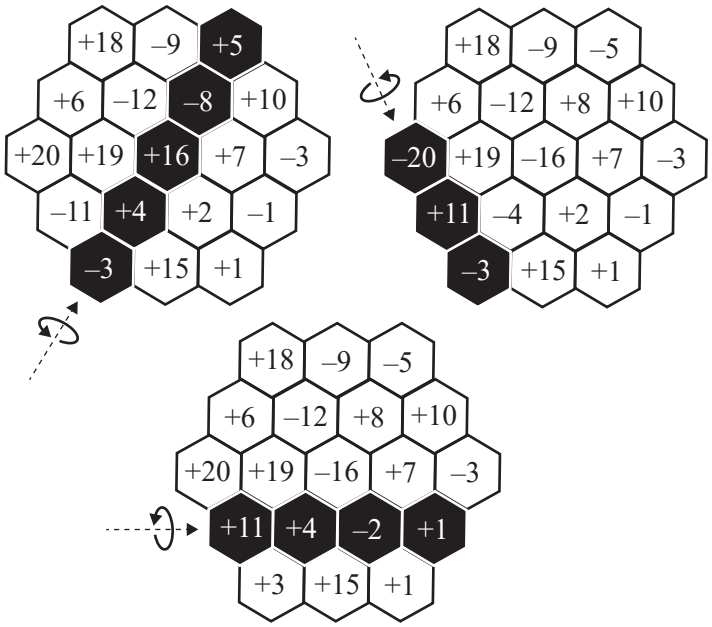
The goal of the game is to maximise the sum of the face-up values on the tiles.

A 'row' refers to a straight line of sequentially adjacent tiles. In the game, a player may flip any row of tiles to its reverse side. Three examples of this are shown in the diagram below. The player may flip any number of rows. It is known that this flip operation does not allow for all possible grid arrangements to be generated.

An example of a small HexaReverso board



Three examples of the flip operation



a. It is important that tiles in a particular row can be efficiently identified.

i. Describe how data about the tiles in a HexaReverso game could be stored. A single ADT or a combination of ADTs may be used. 3 marks

ii. Explain how the flip operation would be performed within the data structure described in **part a.i.** 3 marks

b. **i.** What features of this game indicate that a heuristic approach might be needed to achieve the goal of the game? 2 marks

ii. What would be a limitation of using a heuristic approach? 1 mark

c. Kim is going to apply the simulated annealing meta-heuristic to the goal of this game.

i. Describe how she could generate a new candidate solution from a current solution. 1 mark

ii. Kim has correctly implemented the simulated annealing algorithm, but finds that it only accepts candidate solutions that are improvements on the current solution.

Describe how she could modify the parameters of the algorithm to correct this issue. 1 mark

Question 11 (5 marks)

Consider the following algorithm that might be used to solve a problem where solutions can be randomly generated.

```
soln = generate random solution
temperature = 1
min_temperature = 0.01
n_iterations = 100
while temperature > min_temperature
     for i = 1 to n_iterations
        soln_new = generate neighbouring solution of soln
        if cost(soln) >= cost(soln_new)
            soln = soln_new
        else
            prob = (random 0 to 100)/100
            if e^(cost(soln)-cost(soln_new))/temperature > prob
                soln = soln_new
            endif
        endif
     endfor
     temperature = cooling_factor * temperature
endwhile
```

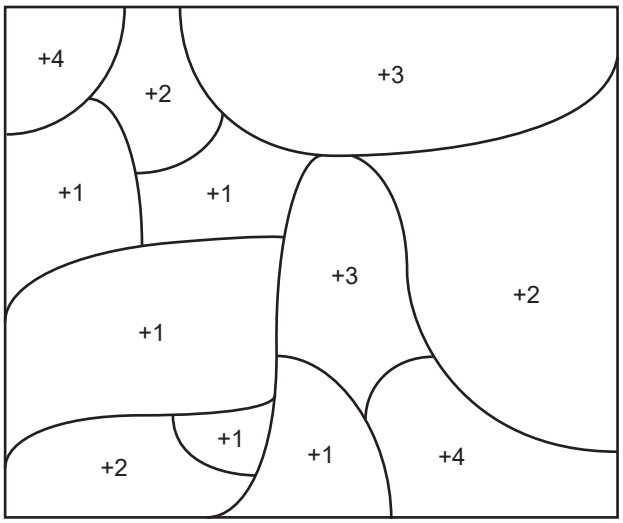
a. State the range of valid values for `cooling_factor`, so that at least 200 random solutions are generated and the algorithm terminates. 2 marks

b. Given that `soln_new` is generated in the neighbourhood of `soln`, why is it a good idea to sometimes replace `soln` with `soln_new` when `cost(soln) < cost(soln_new)`? 1 mark

c. Give **one** example of a problem where a version of the algorithm above is likely to give an acceptable solution. Describe a possible `cost(soln)` for that problem. 2 marks

Question 9 (12 marks)

TerraQuesta is a game played on a board that is divided into regions and each region has a score value. There are many different game boards, and these boards can vary in both size and layout. Larger boards have hundreds of regions. An example of a small board is shown below.



When playing the game, players select regions of the board to own until no further selection is possible. A player cannot select a region that is adjacent to a previously selected region. The game has a single-player version and a two-player version.

Nick, a keen player of TerraQuesta, is trying to beat his previous high score. In the single-player version of the game, the aim of the game is to select regions so that the total score is maximised.

a. Describe a randomised heuristic algorithm that Nick could use to find a set of regions to beat his previous high score. 4 marks

Question 8 (10 marks)

The decision version of the travelling salesman problem asks: given a weighted, undirected graph and a source node, does there exist a path starting and finishing at the source node that visits all remaining nodes exactly once and has a total weight of less than X (where X is a given integer)?

a. Explain why the decision version of the travelling salesman problem is in NP. 2 marks

b. Describe a greedy approach to solving the travelling salesman problem. 2 marks

c. An alternative approach to solving the travelling salesman problem is to use a simulated annealing algorithm. The greedy approach described in **part b.** can be used to generate an initial candidate solution for the simulated annealing algorithm. High-level pseudocode for the simulated annealing algorithm is provided below.

```
Generate initial candidate solution, S
Repeat until a terminating condition is reached:
    Generate a new candidate solution, S_new
    Set a temperature, T
    If the new candidate solution is accepted:
        Set S = S_new
Return S
```

i. Explain how the algorithm would determine whether a new candidate solution is accepted. 3 marks

ii. State a terminating condition that could be used for this algorithm. 1 mark

iii. Describe **one** advantage and **one** limitation of using a simulated annealing algorithm to solve the travelling salesman problem. 2 marks